Deep Learning

E-book Notes for Quizzes & End-Term

Week 9

Learning Vectorial Representations of Words

What you'll learn

1. One-hot vectors and why they fail
2. Distributional hypothesis & co-occurrence matrices
3. PMI, PPMI and SVD-based embeddings
4. Continuous Bag of Words (CBOW)
5. Skip-gram model
6. Negative Sampling & Contrastive Estimation
7. Hierarchical Softmax
8. GloVe representations
9. Evaluating word embeddings
10. SVD vs Word2Vec — the hidden link

Source: CS7015 (IIT Madras) by Prof. Mitesh M. Khapra

Made by Anmol Kansal with AI

April 29, 2026

Contents

1. Why Words Need Vectors

Intuition

Computers don't understand words like "*cat*" or "*dog*". They only understand **numbers**. So before any neural network can do sentiment analysis, translation, or summarisation, we must *convert words into numerical vectors*. The whole of Week 9 is about answering: *what is a good vector for a word?*

Suppose we are given an input stream of words (a sentence, a document) and we want to learn some function of it, say sentiment $\hat{y} = f(\text{words})$. A machine learning algorithm requires its input as a vector $\mathbf{x} \in \mathbb{R}^d$. Therefore the very first step in any NLP pipeline is to choose a **vector representation** for every word.

The pipeline: `words` $\longrightarrow$ **vectorisation** $\longrightarrow$ $\mathbf{x} \in \mathbb{R}^d$ $\longrightarrow$ `model` f $\longrightarrow$ $\hat{y}$

This week we'll explore representations in increasing order of sophistication: *one-hot* $\rightarrow$ *co-occurrence* $\rightarrow$ *SVD* $\rightarrow$ *word2vec* $\rightarrow$ *GloVe*.

2. Module 10.1 — One-Hot Representations

2.1 The set-up

Given a corpus, we form V, the **vocabulary**, the set of all unique words that appear. Let $|V|$ denote the size of the vocabulary.

Worked Example

Toy corpus

- Human machine interface for computer applications
- User opinion of computer system response time
- User interface management system
- System engineering for improved response time

$V = \{$human, machine, interface, for, computer, applications, user, opinion, of, system, response, time, manage
so $|V| = 15$.

2.2 Definition

Definition

A **one-hot vector** for the i-th word in vocabulary V is a vector of size $|V|$ that has a 1 in the i-th position and a 0 everywhere else.

$$\mathbf{e}_i = (\underbrace{0, 0, \ldots, 0}_{i-1 \text{ zeros}}, 1, 0, \ldots, 0) \in \{0, 1\}^{|V|}$$

For example, with the ordering $V = [\text{cat}, \text{dog}, \text{truck}, \ldots]$:

$$\text{cat: } (0,0,0,0,0,1,0) \quad \text{dog: } (0,1,0,0,0,0,0) \quad \text{truck: } (0,0,0,1,0,0,0)$$

2.3 Why one-hot fails — three killer problems

Quiz Alert!

Three flaws of one-hot:

1. $|V|$ is huge (50K words for the Penn TreeBank, 13M for Google 1T) $\Rightarrow$ **very high-dimensional**.
2. It captures **no notion of similarity**. The vectors are mutually orthogonal — cat is just as far from dog as it is from truck.
3. Distance is the **same for every pair** of words.

2.3.1 Provable mathematical facts

For *any* two distinct words w_i, w_j with one-hot vectors $\mathbf{e}_i, \mathbf{e}_j$:

$$\|\mathbf{e}_i - \mathbf{e}_j\|_2 = \sqrt{(1)^2 + (-1)^2} = \sqrt{2}$$

$$\cos(\mathbf{e}_i, \mathbf{e}_j) = \frac{\mathbf{e}_i^\top \mathbf{e}_j}{\|\mathbf{e}_i\| \, \|\mathbf{e}_j\|} = \frac{0}{1 \cdot 1} = 0$$

Every word is equidistant from every other word. "*Cat is as similar to dog as it is to truck*" — clearly absurd.

Memory Trick

Remember the constants: for any pair of distinct one-hot vectors,

$$\boxed{\text{Euclidean distance} = \sqrt{2} \qquad \text{Cosine similarity} = 0}$$

This is a classic quiz question. The Euclidean distance comes from $\sqrt{1^2 + 1^2}$ — one $+1$ from the active position of $\mathbf{e}_i$ and one -1 from the active position of $\mathbf{e}_j$.

3. Module 10.2 — Distributed Representations

3.1 The distributional hypothesis

> *"You shall know a word by the company it keeps."*
> — J. R. Firth, 1957

Intuition

The meaning of a word is determined by the words that surround it. If you keep seeing "*bank*" next to "*financial*", "*deposits*", "*credit*" — those neighbouring words *are* the meaning of bank. Two words are similar if they appear in similar contexts.

This is called the **distributional hypothesis**, and representations built on it are called **distributional** (or **distributed**) representations.

3.2 Co-occurrence matrix

Definition

The **co-occurrence matrix** X is a $|V| \times |V|$ matrix where X_{ij} is the number of times word w_j (the context) appears within a window of size k around word w_i (the term). Also called a **word** $\times$ **context** matrix.

For our toy corpus with $k = 2$ (window of 2 on each side):

	human	machine	system	for	$\cdots$	user
human	0	1	0	1	$\cdots$	0
machine	1	0	0	1	$\cdots$	0
system	0	0	0	1	$\cdots$	2
for	1	1	1	0	$\cdots$	0
$\vdots$					$\ddots$	$\vdots$
user	0	0	2	0	$\cdots$	0

Each **row** is now a vector representation of the word — and crucially, similar words have similar rows!

3.3 Fixable problems

3.3.1 Problem 1: Stop-word domination

Stop words like "*a, the, for*" appear everywhere, blowing up their row counts.

- **Solution 1**: Ignore very frequent words.
- **Solution 2**: Cap the count at a threshold t:

$$X_{ij} = \min\big(\text{count}(w_i, c_j),\, t\big), \quad t \approx 100.$$

3.3.2 Problem 2: Raw counts are misleading

Replace raw counts with **Pointwise Mutual Information (PMI)**:

$$\text{PMI}(w, c) = \log \frac{P(c \mid w)}{P(c)} \tag{1}$$

$$= \log \frac{\text{count}(w, c) \cdot N}{\text{count}(w) \cdot \text{count}(c)}, \tag{2}$$

where N is the total number of words.

Intuition

PMI asks: "*Does c appear with w more often than chance would predict?*" If c and w are independent then $P(c \mid w) = P(c)$ and PMI $= \log 1 = 0$. Positive PMI means *associated*; negative means *anti-associated*.

Issue: if $\text{count}(w, c) = 0$ then PMI $= -\infty$. We use:

$$\text{PMI}_0(w, c) = \begin{cases} \text{PMI}(w, c) & \text{if count}(w, c) > 0 \\ 0 & \text{otherwise} \end{cases}$$

$$\text{PPMI}(w, c) = \max\big(\text{PMI}(w, c),\, 0\big) \quad (\textbf{Positive PMI} \text{ — most common in practice})$$

Memory Trick

PMI mnemonic: "$\log \frac{\text{joint}}{\text{independent}}$". The numerator is the actual joint probability, the denominator is what you'd expect if the two were independent. The ratio is 1 (PMI = 0) when they're independent.

3.4 Severe problems (need a different cure)

Even with PPMI, the matrix is:

- Very high-dimensional ($|V|$ can be millions)
- Very sparse (most entries are 0)
- Grows linearly with vocabulary size

Solution: Apply **dimensionality reduction** via SVD. This leads us to the next module.

Module Summary

- One-hot ⇒ no similarity, $|V|$-dim, sparse.
- Distributional hypothesis ⇒ "a word is its company".
- Co-occurrence ⇒ rows of an $|V| \times |V|$ matrix.
- Raw counts overweight stop words ⇒ use PMI / PPMI.
- Still too big and sparse ⇒ next: **SVD**.

4. Module 10.3 — SVD for Word Representations

4.1 What SVD does

Definition

Singular Value Decomposition (SVD). Any real matrix $X \in \mathbb{R}^{m \times n}$ can be factored as

$$X = U \Sigma V^\top$$

where

- $U \in \mathbb{R}^{m \times m}$ has orthonormal columns (left singular vectors).
- $\Sigma \in \mathbb{R}^{m \times n}$ is diagonal with singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$.
- $V \in \mathbb{R}^{n \times n}$ has orthonormal columns (right singular vectors).

For word embeddings we apply SVD to $X = X_{\text{PPMI}}$ (the PPMI co-occurrence matrix).

4.2 Rank-k approximation

The rank-k truncated SVD is:

$$X \approx U_{m \times k} \Sigma_{k \times k} V^\top_{k \times n} = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^\top + \sigma_2 \mathbf{u}_2 \mathbf{v}_2^\top + \cdots + \sigma_k \mathbf{u}_k \mathbf{v}_k^\top$$

So the original matrix is written as a **sum of k rank-1 matrices**. Each $\sigma_i \mathbf{u}_i \mathbf{v}_i^\top$ is in $\mathbb{R}^{m \times n}$ because it's an outer product of an $m \times 1$ vector with a $1 \times n$ vector.

Eckart–Young theorem (just remember the punchline):
Truncated SVD gives the best rank-k approximation of X in Frobenius norm.

4.3 Why does this give meaningful word vectors?

Intuition

SVD finds the **latent dimensions** of the data — directions in which most of the "co-occurrence variance" lies. Two words that appeared in many similar contexts will end up close in this lower-dimensional space, even if they were never adjacent to each other in the corpus. SVD effectively *discovers latent semantics*.

4.4 The word and context vectors

Once we compute $X \approx U\Sigma V^\top$:

- **Word representation** (most common choice): $W_{\text{word}} = U\Sigma \in \mathbb{R}^{m\times k}$, so the i-th row of $U\Sigma$ is the k-dim embedding of word w_i.
- **Context representation**: $W_{\text{context}} = V \in \mathbb{R}^{n\times k}$.

4.5 Higher-order co-occurrences (the secret sauce)

A surprising bonus: SVD captures **higher-order co-occurrences**. Even if two words never appeared in the same window, if they appeared with the *same* third word, SVD will pull them closer.

Worked Example

"*System*" and "*machine*" may never appear together. But both appear with "*user*". Truncated SVD will give them similar vectors. That's why *system* and *machine* end up near each other in the latent space.

Memory Trick

Why SVD beats PPMI alone:

- PPMI captures only **first-order** (direct) co-occurrence.
- SVD of PPMI captures **higher-order** (transitive) co-occurrence.

4.6 Choosing k

Rule of thumb: $k = 50$ to 300. Smaller k means more compression but more loss.

Module Summary

SVD pipeline: corpus → co-occurrence matrix → PPMI matrix → SVD, take top k singular components → word vectors $W = U\Sigma$.

5. Module 10.4 — Continuous Bag of Words (CBOW)

5.1 Motivation: predict, don't count

Intuition

SVD takes the entire corpus, builds a giant matrix, then factorises it. That is a **count-based** approach.

Word2Vec (Mikolov et al., 2013) takes a different route: **predict-based**. Set up a tiny neural network that, given context words, predicts the target word. The *weights* of the trained network become the word vectors. No giant matrix needed.

There are **two flavours** of Word2Vec:

- **CBOW**: predict centre word given context (this module).
- **Skip-gram**: predict context given centre word (next module).

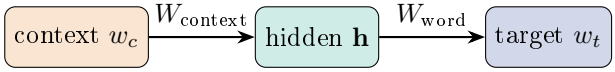
5.2 The set-up

We use a window of $\pm d$ words around a target word. For window size $d = 2$:

$$\underbrace{\cdots \; w_{t-2} \; w_{t-1}}_{\text{left context}} \; \boxed{w_t} \; \underbrace{w_{t+1} \; w_{t+2} \; \cdots}_{\text{right context}}$$

CBOW reads: "given $w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2}$, predict w_t".

5.3 The simplest case: one context word



Architecture:

- Input layer: one-hot vector $\mathbf{x} \in \mathbb{R}^{|V|}$ of the context word.
- Embedding (input-to-hidden) matrix: $W_{\text{context}} \in \mathbb{R}^{|V| \times k}$ (some texts call it W).
- Hidden representation: $\mathbf{h} = W_{\text{context}}^{\top} \mathbf{x} \in \mathbb{R}^k$. Since $\mathbf{x}$ is one-hot, $\mathbf{h}$ is just the row of W_{context} corresponding to the context word.
- Output (hidden-to-output) matrix: $W_{\text{word}} \in \mathbb{R}^{k \times |V|}$ (often denoted W').
- Output: $\mathbf{u} = W_{\text{word}}^{\top} \mathbf{h}$, then $\hat{\mathbf{y}} = \text{softmax}(\mathbf{u})$.

5.4 The forward pass with multiple context words

When the context contains several words $w_{c_1}, \ldots, w_{c_C}$, we **average** their embeddings:

$$\mathbf{h} \;=\; \frac{1}{C}\big(W_{\text{context}}^{\top}\mathbf{x}_{c_1} + \cdots + W_{\text{context}}^{\top}\mathbf{x}_{c_C}\big) \;=\; \frac{1}{C}\sum_{j=1}^{C} \mathbf{v}_{c_j},$$

where $\mathbf{v}_{c_j}$ is the c_j-th row of W_{context}.

Then for the target word w_t:

$$\hat{y}_t = P(w_t \mid \text{context}) = \frac{\exp(\mathbf{u}_{w_t}^{\top}\mathbf{h})}{\sum_{j=1}^{|V|} \exp(\mathbf{u}_j^{\top}\mathbf{h})}. \tag{3}$$

Here $\mathbf{u}_j$ is the j-th column of W_{word} (i.e., the "output" embedding of word w_j).

5.5 The loss function

We use cross-entropy with the one-hot target $\mathbf{y}$:

$$\mathcal{L} \;=\; -\log P(w_t \mid \text{context}) \;=\; -\log \hat{y}_t.$$

For one training pair:

$$\boxed{\mathcal{L} = -\mathbf{u}_{w_t}^{\top}\mathbf{h} + \log \sum_{j=1}^{|V|} \exp(\mathbf{u}_j^{\top}\mathbf{h})}$$

5.6 Two embeddings per word

Quiz Alert!

After training, every word has **two** learned vectors:

- Its row in W_{context} (when it acts as context)
- Its column in W_{word} (when it acts as target)

The final "word embedding" is usually taken as the row of W_{context}, but you can also average the two or concatenate.

5.7 Backpropagation in CBOW (intuition)

The update rules push:

- The hidden vector $\mathbf{h}$ *toward* the output embedding of the correct word.
- All other output embeddings *away* from $\mathbf{h}$.

Per-step the cost of the softmax is $O(|V|)$ — we'll fix this in Modules 10.6 and 10.7.

5.8 Worked numerical example (PYQ-style)

Worked Example

CBOW with embedding dimension 2. Vocabulary {"one","two","three","four"}, window size 1 (only previous word as context). The current matrices are

$$W_{\text{word}} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 1 & 2 & -2 \end{bmatrix}, \qquad W_{\text{context}} = \begin{bmatrix} 0 & 1 & 0 & 1 \\ -1 & 0 & 1 & 1 \end{bmatrix}.$$

Columns are indexed in order one, two, three, four. The current sample is "two three" (i.e. context = "two", target = "three").

Step 1 — Hidden vector. The context is "two" (column 2). With one context word, $\mathbf{h}$ equals that column of W_{context}:

$$\mathbf{h} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

Step 2 — Scores $\mathbf{u} = W_{\text{word}}^{\top}\mathbf{h}$:

$$\mathbf{u} = \begin{bmatrix} 1 & -1 \\ 1 & 1 \\ 1 & 2 \\ 1 & -2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}.$$

Step 3 — Softmax over scores:

$$P(w \mid \text{"two"}) = \frac{e^{u_w}}{\sum_j e^{u_j}} = \frac{e}{4e} = \tfrac{1}{4} = 0.25$$

for every word. So $P(\text{"three"} \mid \text{"two"}) = 0.25$.

Quiz Alert!

Quiz tip: when all logits are equal, softmax is uniform: $\frac{1}{|V|}$. This is exactly the kind of setup used in PYQ end-term questions.

Module Summary

CBOW = *average of context embeddings* → *softmax over* $|V|$ → *cross-entropy loss.* Two matrices, two embeddings per word, one $|V|$-way softmax that we'll soon need to speed up.

6. Module 10.5 — Skip-Gram Model

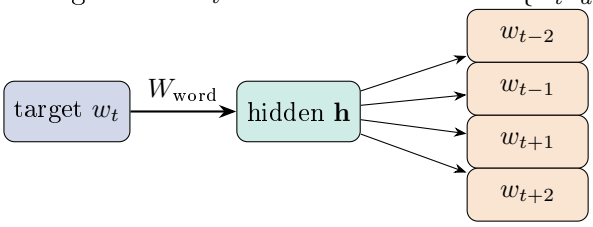
6.1 Skip-gram in one sentence

Intuition

Skip-gram is CBOW *turned around*: instead of using context to predict the centre word, we use the **centre word to predict each surrounding context word**. If CBOW asks "given the neighbours, what is the missing word?", skip-gram asks "given this word, what neighbours would you expect?".

6.2 Architecture

For a target word w_t and a context window $\{w_{t-d}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+d}\}$:



- Input: one-hot of *target* word $\mathbf{x}_t \in \mathbb{R}^{|V|}$.
- Hidden: $\mathbf{h} = W_{\text{word}}^{\top}\mathbf{x}_t$ — i.e., the row of W_{word} for w_t.
- Output: *the same* softmax used $2d$ times to predict each context word.

  The probability of context word w_c given target w_t:

$$P(w_c \mid w_t) = \frac{\exp(\mathbf{u}_c^{\top}\mathbf{v}_t)}{\sum_{j=1}^{|V|}\exp(\mathbf{u}_j^{\top}\mathbf{v}_t)}, \tag{4}$$

where $\mathbf{v}_t$ is the row of W_{word} for w_t and $\mathbf{u}_c$ is the column of W_{context} for context word w_c.

6.3 Loss for skip-gram

We assume context words are independent given the centre word, so the loss for one centre word and its $2d$ context words is:

$$\mathcal{L} = -\sum_{c \in \text{context}(t)} \log P(w_c \mid w_t).$$

6.4 CBOW vs Skip-gram — the cheat sheet

	CBOW	**Skip-gram**
What predicts what?	context ⇒ centre	centre ⇒ context
Number of training samples per window	1	$2d$
Faster training	**Yes** (one softmax/window)	No
Better for rare words	No	**Yes** (rare word becomes the input many times)
Better with large data	**Slight edge**	Slight edge with smaller data

Memory Trick

Mnemonic:
CBOW = **C**ontext predicts centre.
Skip-gram = **S**ingle word skips ahead to predict its neighbours.

6.5 The expensive softmax

Both models have a hidden-to-output weight matrix of size $k \times |V|$. Computing the denominator of the softmax requires summing over the entire vocabulary — $O(|V|)$ per training example. For $|V| = 10^6$ this is prohibitive. Modules 10.6 and 10.7 fix this.

7. Module 10.6 — Contrastive (Negative) Sampling

7.1 The trick: turn classification on its head

Intuition

Instead of asking "*out of all $|V|$ words, which one is the context?*", ask the simpler binary question: "*is this word a true context of the target, or not?*". Replace one expensive $|V|$-way softmax with many cheap binary classifications.

7.2 Construction

1. For each (target, true-context) pair (w, c), sample K "negative" (non-context) words $w_1^-, \ldots, w_K^-$ from a noise distribution $P_n(w)$.
2. Train a logistic regression: predict 1 for the true pair (w, c) and 0 for each negative pair (w, w_k^-).

   The objective for a single positive pair becomes:

$$\mathcal{L}_{\text{NEG}} = -\log \sigma(\mathbf{u}_c^\top \mathbf{v}_w) \; - \; \sum_{k=1}^{K} \log \sigma(-\mathbf{u}_{w_k^-}^\top \mathbf{v}_w), \tag{5}$$

where $\sigma(x) = 1/(1 + e^{-x})$.

- The first term **pushes up** the score of the true pair.
- The summation **pushes down** the score of negative pairs.

7.3 Choice of the noise distribution

Mikolov et al. found that

$$P_n(w) \; \propto \; \text{count}(w)^{3/4}$$

works well in practice. The 3/4 exponent damps the dominance of frequent words while still preferring them over rare ones.

Quiz Alert!

Why 3/4? It's empirical, not derived. Just remember: *unigram distribution raised to the power* $\frac{3}{4}$.

7.4 Speed-up

Cost per example drops from $O(|V|)$ to $O(K + 1)$ where K is typically 5–20. This is the version of word2vec that's actually used in practice.

7.5 Why is it called "contrastive"?

We are explicitly contrasting the true word against K noise words. The model learns a representation that **discriminates** true context from random words.

Module Summary

Negative sampling replaces the expensive softmax with $K+1$ binary logistic classifications. Sample negatives from $P_n(w) \propto \text{count}(w)^{3/4}$. This is the engine that makes word2vec scale.

8. Module 10.7 — Hierarchical Softmax

8.1 Idea

Intuition

Instead of a flat $|V|$-way decision, organise the vocabulary as the leaves of a **binary tree**. Predicting a word becomes a sequence of $O(\log |V|)$ left/right decisions down the tree. Each internal node is a tiny binary classifier.

8.2 The maths

Let $L(w)$ be the path length from root to word w, and let $n(w, j)$ be the j-th node on that path. At each internal node n store a vector $\mathbf{v}_n$.

The probability of word w given hidden vector $\mathbf{h}$ is:

$$P(w \mid \mathbf{h}) \;=\; \prod_{j=1}^{L(w)-1} \sigma\big(\,\text{sign}\big(n(w, j+1) \text{ is left child}\big) \cdot \mathbf{v}_{n(w,j)}^{\top}\mathbf{h}\,\big).$$

- At each node, $\sigma(\mathbf{v}_n^{\top}\mathbf{h})$ is the probability of going *left*.
- The probability of reaching leaf w is the product of all the chosen branch probabilities along the path.
- Crucially, $\sum_w P(w \mid \mathbf{h}) = 1$ because the tree partitions the vocabulary.

8.3 Speed-up

A balanced binary tree has $\log_2 |V|$ levels, so each prediction costs $O(\log_2 |V|)$. For $|V| = 10^6$ that's only ≈ 20 classifications instead of a million.

8.4 What tree to use?

Practically, a **Huffman tree** built from word frequencies. Frequent words land at shallow depths $\Rightarrow$ even faster.

Memory Trick

Two ways to dodge the $|V|$-way softmax:

Negative Sampling vs. Hierarchical Softmax

Both trade exactness for speed. Negative sampling is more popular in modern code; hierarchical softmax was the original speed-up in Mikolov et al.

Module Summary

Hierarchical softmax converts a flat $|V|$-way decision into a path of $O(\log_2 |V|)$ binary decisions through a (Huffman) tree.

9. Module 10.8 — GloVe Representations

9.1 The big idea

Intuition

Word2Vec is predict-based but local (window). SVD is count-based but global (matrix). **GloVe** (Pennington, Socher & Manning, 2014) tries to be both: it factorises a global co-occurrence matrix using ratios of co-occurrence probabilities, motivated by the observation that *ratios* of probabilities encode meaning more cleanly than raw probabilities.

9.2 Co-occurrence ratios

Let X_{ij} be the number of times word j appears in the context of word i, and $P_{ij} = P(j \mid i) = X_{ij}/X_i$ where $X_i = \sum_k X_{ik}$.

Key insight: the *ratio* $\frac{P_{ik}}{P_{jk}}$ is large when k is associated with i but not j, small when associated with j but not i, and near 1 when associated with both or neither.

Worked Example

$i = \text{ice}, j = \text{steam}$.

- $k = \text{solid}$: $P_{\text{ice,solid}}/P_{\text{steam,solid}} \gg 1$.
- $k = \text{gas}$: $\ll 1$.
- $k = \text{water}$: ≈ 1 (related to both).
- $k = \text{fashion}$: ≈ 1 (related to neither).

9.3 The GloVe objective

We seek vectors $\mathbf{w}_i$, $\tilde{\mathbf{w}}_j$ and biases $b_i, \tilde{b}_j$ such that

$$\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j \;\approx\; \log X_{ij}.$$

The least-squares loss with a weighting function $f(X_{ij})$ to dampen frequent terms is

$$\boxed{\mathcal{L} \;=\; \sum_{i,j=1}^{|V|} f(X_{ij})\big(\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij}\big)^2}$$

9.4 The weighting function

The standard choice is

$$f(x) = \begin{cases} (x/x_{\max})^\alpha & x < x_{\max} \\ 1 & x \geq x_{\max} \end{cases}$$

with $\alpha = \frac{3}{4}$ and $x_{\max} = 100$ (familiar 3/4 again!).

Properties:

- $f(0) = 0$ — never train on zero co-occurrences.
- Monotonically increasing — rare pairs get small weight.
- Saturates at $x_{\max}$ — frequent pairs do not dominate.

9.5 GloVe vs Word2Vec vs SVD — at a glance

	SVD on PPMI	Word2Vec	GloVe
Type	count-based	predict-based	both
Uses global statistics	yes	no (local windows)	yes
Optimised by	matrix factorisation	SGD on local windows	SGD on global pairs
Captures higher-order?	yes (latent dims)	yes (via predictions)	yes

Memory Trick

One-line GloVe: "*Make the dot product of two word vectors equal to the log of how often they co-occur.*"

10. Module 10.9 — Evaluating Word Representations

10.1 Two flavours of evaluation

- **Intrinsic**: directly probe what the embeddings "know".
- **Extrinsic**: plug them into a downstream task (NER, parsing, sentiment) and measure task performance.

10.2 Intrinsic evaluation tasks

10.2.1 (a) Semantic relatedness / similarity

Datasets like WordSim-353 contain pairs of words with human-rated similarity scores. Compute cosine similarity between each pair's embeddings, then take the **Spearman rank correlation** with human ratings.

10.2.2 (b) Synonym detection

Given a target word, choose the synonym from a list of candidates by nearest-neighbour cosine similarity.

10.2.3 (c) Analogy tasks

The most famous result of word2vec:

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}.$$

We solve $a : b :: c :?$ by computing $\arg\max_d \cos(\mathbf{v}_d, \mathbf{v}_b - \mathbf{v}_a + \mathbf{v}_c)$.

Intuition

The vector difference `king` − `man` encodes *royalty* along that direction. Adding it to `woman` gives `queen`. Embeddings learn directions for relations like gender, tense, capital-city, etc.

10.2.4 (d) Categorisation / clustering

Cluster the embeddings; check whether the clusters align with known semantic categories.

10.3 Extrinsic evaluation

Use the embeddings as the input layer of a model for a downstream task and report the task metric (F1, accuracy, BLEU, etc.). This is the gold standard for *practical* usefulness.

10.4 Common pitfalls

Common Mistake

A model that performs well on intrinsic tasks can still fail on a downstream task. Always ultimately validate on the task you care about.

11. Module 10.10 — Relation Between SVD and Word2Vec

11.1 The surprising connection

Intuition

Levy and Goldberg (2014) showed that **Skip-gram with negative sampling (SGNS) is implicitly factorising a shifted PMI matrix**. So word2vec, the supposed "new" predict-based method, is mathematically very close to the old SVD-on-PPMI approach.

11.2 The result

For a target w and context c trained with K negative samples, the optimum satisfies

$$\boxed{\mathbf{w}^\top \mathbf{c} \;=\; \mathrm{PMI}(w, c) - \log K}$$

That is, word2vec finds vectors whose dot product equals **shifted PMI**. The matrix M with $M_{ij} = \mathrm{PMI}(w_i, c_j) - \log K$ is being implicitly factored as $M = W\,C^\top$.

11.3 Punchline

The two camps converge:

- Count-based (SVD on PPMI) and predict-based (SGNS) are doing essentially the same thing.
- The difference is only in the loss function, the weighting, and the optimisation algorithm.
- Both end up factoring a (P)PMI-like matrix.

Memory Trick

Quiz favourite: "*Skip-gram with negative sampling implicitly factorises which matrix?*"
Answer: **the shifted PMI matrix**, $\mathrm{PMI}(w, c) - \log K$.

12. Practice Questions with Answers

This section combines questions from past end-term papers (PYQs) with additional practice questions in the style of the IIT M Deep Learning quizzes.

12.1 From the 2025 April end-term paper (relevant questions only)

PYQ – Apr 2025 – Q27/Q28: CBOW

Consider the CBOW model for learning word embeddings with embedding dimension 2. Window size 1 (only previous word as context). Vocabulary {"one", "two", "three", "four"}. The first column of each matrix is the embedding for "one", and so on:

$$W_{\text{word}} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 1 & 2 & -2 \end{bmatrix}, \quad W_{\text{context}} = \begin{bmatrix} 0 & 1 & 0 & 1 \\ -1 & 0 & 1 & 1 \end{bmatrix}.$$

Sample at this step: "two three". "three" is the true label.
Q27 (Q28 in paper). Find $P(\text{three} \mid \text{two})$.

Solution. The hidden vector is the column of W_{context} for "two" (column 2): $\mathbf{h} = (1, 0)^\top$. Scores $\mathbf{u} = W_{\text{word}}^\top \mathbf{h} = (1, 1, 1, 1)^\top$. Softmax of $(1, 1, 1, 1)$ gives $(0.25, 0.25, 0.25, 0.25)$. So $P(\text{three} \mid \text{two}) = \boxed{0.25}$. (Accepted: 0.2 to 0.3.)

PYQ – Apr 2025 – Q29: One-step gradient update

Using the same sample, perform one update of gradient descent with $\eta = 1$ for the word embedding of "three". If the updated embedding of "three" (in W_{word}) is (a, b), enter $b - a$.

Solution. The gradient of cross-entropy loss with respect to $\mathbf{u}_t$ (the column of W_{word} for the target "three") is

$$\frac{\partial \mathcal{L}}{\partial \mathbf{u}_t} = (\hat{y}_t - 1)\mathbf{h}.$$

The current embedding for "three" is column 3 of W_{word}: $(1, 2)^\top$. With $\hat{y}_t = 0.25$, the update is

$$\mathbf{u}_t^{\text{new}} = \mathbf{u}_t - \eta(\hat{y}_t - 1)\mathbf{h} = (1, 2)^\top - (0.25 - 1)(1, 0)^\top = (1, 2)^\top + (0.75, 0)^\top = (1.75, 2)^\top.$$

So $a = 1.75$, $b = 2$, and $b - a = \boxed{0.25}$. (Accepted: 0.2 to 0.3.)

Quiz Alert!

Why the trick of asking $b - a$? It eliminates rounding errors in any intermediate step that adds the same scalar to both components.

12.2 Additional practice questions

12.2.1 Q1 (one-hot mathematics)

Two distinct words w_i and w_j are encoded as one-hot vectors. What is their (a) Euclidean distance and (b) cosine similarity?

Answer. (a) $\sqrt{2}$. (b) 0. This holds for *any* pair of distinct one-hot vectors.

12.2.2 Q2 (PMI sign)

If two words w and c are statistically independent, what is $\text{PMI}(w, c)$?

Answer. $\text{PMI} = \log \frac{P(c|w)}{P(c)}$. Independence implies $P(c \mid w) = P(c)$, so the ratio is 1 and $\log 1 = 0$. Answer: **0**.

12.2.3 Q3 (PPMI matrix)

For the toy corpus with PMI values, $\text{PMI}(\text{human}, \text{system}) = -\infty$ because they never co-occur. What is the corresponding entry in the PPMI matrix?

Answer. PPMI clips negative (and undefined) values to 0. So $\text{PPMI}(\text{human}, \text{system}) = \mathbf{0}$.

12.2.4 Q4 (SVD dimensions)

Given a co-occurrence matrix $X \in \mathbb{R}^{50000\times50000}$, after rank-100 truncated SVD what are the dimensions of the word embedding matrix $W = U\Sigma$?

Answer. $U \in \mathbb{R}^{50000\times100}$, $\Sigma \in \mathbb{R}^{100\times100}$, so $W \in \mathbb{R}^{50000\times100}$. Each word becomes a 100-dimensional vector.

12.2.5 Q5 (CBOW vs Skip-gram per-window samples)

For a window of size d on each side, how many (input, target) training samples does (a) CBOW and (b) Skip-gram produce per centre word?

Answer. (a) CBOW: 1 sample per centre word. (b) Skip-gram: $2d$ samples (one for each context word).

12.2.6 Q6 (negative sampling cost)

Word2Vec with vocabulary size $|V| = 10^6$. Compare the per-step cost of full softmax vs negative sampling with $K = 10$ negatives.

Answer. Full softmax: $O(|V|) = 10^6$ dot products. Negative sampling: $O(K + 1) = 11$ sigmoid evaluations. Speed-up of about 10^5.

12.2.7 Q7 (the 3/4 exponent)

In word2vec's negative sampling, words are drawn from a noise distribution $P_n(w) \propto \text{count}(w)^?$. What is the exponent?

Answer. $\boxed{3/4}$. Empirical, not derived. Suppresses overly frequent words while still preferring common over rare.

12.2.8 Q8 (hierarchical softmax tree)

For a vocabulary of size $|V| = 2^{20}$ ($\approx$ 1M), at most how many sigmoid computations does hierarchical softmax need to evaluate $P(w \mid \mathbf{h})$ for one word?

Answer. A balanced binary tree has depth $\log_2 |V| = 20$. So at most $\boxed{20}$ sigmoid evaluations.

12.2.9 Q9 (analogy)

We solve the analogy "man : king :: woman : ?" using word vectors by computing $\mathbf{v} = ?$ and finding the nearest neighbour. Fill in the blank.

Answer. $\mathbf{v} = \mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}}$. The expected nearest neighbour is `queen`.

12.2.10 Q10 (GloVe)

What does the GloVe loss function force the dot product $\mathbf{w}_i^\top \tilde{\mathbf{w}}_j$ to approximate?

Answer. $\log X_{ij}$, the log of the co-occurrence count of w_i and w_j (offset by biases).

12.2.11 Q11 (GloVe weighting)

Why is the weighting $f(X_{ij})$ in GloVe set to 0 when $X_{ij} = 0$?

Answer. If two words never co-occur ($X_{ij} = 0$) then $\log X_{ij} = -\infty$ — undefined. Setting $f(0) = 0$ removes those pairs from the loss entirely.

12.2.12 Q12 (SVD vs SGNS)

Levy and Goldberg's celebrated result: skip-gram with negative sampling implicitly factorises which matrix?

Answer. The shifted PMI matrix: $M_{ij} = \text{PMI}(w_i, c_j) - \log K$, where K is the number of negative samples.

12.2.13 Q13 (intrinsic vs extrinsic)

You report 0.71 Spearman correlation on WordSim-353 but only 0.62 F1 on a downstream NER task. Which evaluation is more meaningful in practice?

Answer. Extrinsic. Intrinsic scores are convenient but ultimately you care about how the embeddings perform on the actual task.

12.2.14 Q14 (CBOW gradient direction)

After one step of gradient descent on a (context, target) pair, in what direction does the output embedding $\mathbf{u}_t$ of the *true* target move?

Answer. It moves in the direction of $+\mathbf{h}$ (since gradient is $(\hat{y}_t - 1)\mathbf{h}$ which is negative, and $\mathbf{u}_t \leftarrow \mathbf{u}_t - \eta(\hat{y}_t - 1)\mathbf{h}$). Intuitively: *the true word's embedding gets pulled toward the hidden vector* $\mathbf{h}$.

12.2.15 Q15 (higher-order co-occurrence)

Two words `system` and `machine` never appear in the same window. After SVD on the PPMI matrix, will their embeddings be similar?

Answer. Possibly yes! If they appear with the *same* third word (e.g., both with "user"), SVD's latent dimensions can pull them close. This is the higher-order co-occurrence effect.

13. Last-Minute Cheatsheet

One-page revision

- **One-hot**: $|V|$-dim, all pairs at distance $\sqrt{2}$, cos sim 0.
- **Co-occurrence**: rows of word×context matrix. Stop words inflate counts; cap or use PMI.
- **PMI/PPMI**: $\log \frac{p(c|w)}{p(c)}$, 0 when independent. PPMI = $\max(\text{PMI}, 0)$.
- **SVD**: $X \approx U\Sigma V^\top$. Word vec $= U\Sigma$. Captures higher-order co-occurrence.
- **CBOW**: context → centre. Average context embeddings → softmax.
- **Skip-gram**: centre → context. $2d$ training pairs per window.
- **Negative sampling**: K binary negatives drawn from $P_n \propto \text{count}^{3/4}$. Cost $O(K{+}1)$.
- **Hierarchical softmax**: cost $O(\log |V|)$ via Huffman tree.
- **GloVe**: $\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j \approx \log X_{ij}$. Weight by $(x/x_{\max})^{3/4}$.
- **Evaluation**: intrinsic (similarity, analogy) and extrinsic (task F1).
- **SGNS = shifted-PMI factorisation**: $\mathbf{w}^\top \mathbf{c} = \text{PMI} - \log K$.

Five formulas to memorise

1. One-hot stats: $d_{euclid} = \sqrt{2}$, $\cos = 0$.
2. PMI: $\log \frac{count(w,c) \cdot N}{count(w) \cdot count(c)}$.
3. CBOW prediction: $P(w_t \mid c) = \dfrac{e^{\mathbf{u}_{w_t}^\top \mathbf{h}}}{\sum_j e^{\mathbf{u}_j^\top \mathbf{h}}}$.
4. Negative sampling loss: $-\log \sigma(\mathbf{u}_c^\top \mathbf{v}_w) - \sum_k \log \sigma(-\mathbf{u}_k^\top \mathbf{v}_w)$.
5. GloVe target: $\mathbf{w}_i^\top \tilde{\mathbf{w}}_j \approx \log X_{ij}$.

End of Week 9 — Word Representations

Made by Anmol Kansal with AI